Extensib1e Math Functions for C++

Document #:	P4188R1
Date:	2026-03-30
Project:	Programming Language C++
Audience:	SG6
	SG18
	SG20
	LEWG

Reply-to: Stéphane Gros-Lemesre

<stephane.groslemesre@gmail.com>

Abstract

This paper proposes making standard library mathematical functions extensible to user-defined types via ADL. To preserve backward compatibility, the proposed approach introduces a new `std::math` sub-namespace. The new namespace also offers an opportunity to introduce a C++-idiomatic math library consistent with general standard library naming patterns.

Contents

1 Motivation

1.1 Problem

User-defined types are central to C++ programming. The standard library inherited from the Standard Template Library its foundational principle of separating data structures and algorithms through generic programming. Containers, iterators, Customization Point Objects (CPOs), custom operators all contribute to enable a natural integration of any type, user-defined or not, in a common ecosystem.

Mathematical functions are a notable gap in this picture. Unlike operators, they are not part of overload resolution in a way that extends to user-defined types, and unlike CPOs, there is no standard hook to participate in. An author implementing an arithmetic-like type will find it easy to implement binary operations such as addition, subtraction, multiplication, and division; comparisons, hashing, or integration with containers for their user-defined type, until they face the problem of implementing the square-root operation or similar mathematical function.

The fundamental problem with defining mathematical functions for a custom type is that because there are no well-defined contract like for the operators, and no CPO in the standard library for that purpose, the mechanism for their integration is not in the hands of the author. A custom implementation can be offered, but whether it will be picked up by generic code depends entirely on how that code was implemented.

This is not what good separation of concerns looks like: for the author of numeric types, providing custom mathematical functions is a leap of faith, while for generic code authors, supporting custom types might not be the primary concern and requires a significant amount of boilerplate.

1.1.1 Naive call sites

It is difficult to quantify the amount of generic code calling mathematical functions without support for user-defined numeric types. This is made even more difficult as these functions can live in the global namespace if `<math.h>` is included. Using GitHub search functionality shows 115k C++ files using the idiom `using std::pow;` for 3.5M C++ files calling `pow(`, of which 741k C++ files calling explicitly `std::pow(`.

Although these numbers have to be taken with caution, as there are other ways to leverage ADL for the `pow` function than `using std::pow;` specifically, and some of the qualified `std::pow` calls might come from non-generic code, it seems likely that a non-negligible amount of generic code does not allow custom types because of this extensibility limitation.

The library **nholthaus/units**, for instance, provides `units::math::sqrt` which calls `std::sqrt` directly on the underlying scalar value, which means it does not enable ADL resolution and thus does not extend to custom underlying types. [nholthaus-units]

This should not be surprising: the responsibility of making custom types work intuitively should belong to their authors, not to their users or third-party. It takes conscious effort for a generic library author to anticipate the needs for wrappers of hypothetical custom types and explicitly provide support for them.

Yet, the consequence of this status quo is substantial: the successful integration of a custom-type in a code-base no longer depends on its own design and implementation, but on the implementation of every other piece of code that will manipulate it. It is effectively a dependency of that type on all of the generic code it interacts and will interact with in the future. Such open-ended, long-term consequences are often unacceptable.

This is significant because custom numeric types can be used to support **correctness and safety** by encoding constraints that are enforced by the type system automatically. This pattern is similar to *newtypes* in Rust, and is sometimes referred to as "semantic-typing". Whether it is by preventing inappropriate conversion or ensuring unit consistency, it can prevent entire classes of bugs and is a well-understood tool for writing safer numeric code. As it stands, its practicality in C++ is significantly restrained by how well such custom types compose with the broader ecosystem, and mathematical functions are a substantial weak point in that composition.

1.1.2 The `using`-statement workaround

The idiomatic workaround is to enable ADL via a `using` declaration:

```
using std::sqrt;
return sqrt(x);
```

This allows a user-defined mathematical function in the same namespace as the type passed into the function to be found via ADL, while falling back to the `std`-defined version for built-in types. However, this idiom has a critical limitation: it requires a statement, and is therefore unavailable in contexts that only accept expressions.

1.1.2.1 Statement-only

Constructor member initializer lists:

```
MyType(A aSq, B b, C c)
    : a(sqrt(aSq)),   // std::sqrt not found through conversion
      b(b),
      c(abs(c))     // std::abs not found through conversion
{}
```

The same applies to default member initializers:

```
struct Foo {
    double x = sqrt(v);  // std::sqrt not found through conversion
};
```

`requires` expressions:

```
template<typename T>
concept numeric = requires(T x) {
    { abs(x) };  // requirement fails
};
```

There are other, less obvious situations where the same limitation applies, such as constant expressions (array size from a new-type) and template arguments.

In all of these cases, the programmer faces the same three unsatisfactory choices:

Option 1: Call `std::` explicitly

```
MyType(A aSq, B b, C c)
    : a(std::sqrt(aSq)),  // silently breaks extensibility
      b(b),
      c(std::abs(c))
{}
```

This compiles and works for built-in types, but silently cuts off any user-defined overload.

Option 2: Restructure to allow a statement

For member initializer lists, this means moving initialization to the constructor body:

```
MyType(A aSq, B b, C c)
    : b(b)
{
    using namespace std;
    this->a = sqrt(aSq);  // requires a to be default-constructible
    this->c = abs(c);   // loses const and reference member support
}
```

This restores ADL but members can no longer be `const` or references, and all members must be default-constructible. Not all contexts can be restructured this way.

Option 3: Write boilerplate helper functions

```
template<typename T> auto my_sqrt(T&& x) {
    using std::sqrt;
    return sqrt(std::forward<T>(x));
}

MyType(A aSq, B b, C c)
    : a(my_sqrt(aSq)),
      b(b),
      c(my_abs(c))
{}
```

This restores extensibility but requires every author of generic code to write and maintain their own dispatch layer; one wrapper per math function, 45+ at least, and likely more in the future (see [P3935R1]).

The ownership of these wrappers is unclear. Custom numeric-type authors should probably embed them in their math function implementations, but authors of generic code using such functions cannot rely on this being provided and may need to implement them redundantly to support a wider range of types.

1.1.2.2 Existing Practice

The existence of multiple widely-used C++ libraries implementing exactly this machinery is evidence that there are real use-cases and that the status quo is encouraging unnecessary duplication.

mp-units, **Eigen**, and **Boost.Units** have all independently converged on the same core mechanism: bring `std::sqrt` into scope and let ADL do the work.

```
using std::sqrt;
return sqrt(x);
```

The surrounding machinery differs:

— mp-units adds an explicit member function check. [mp-units]

— Eigen wraps the dispatch in a traits struct with SIMD specialisations and relies on macros to minimize the duplication between different functions. [eigen-math]

```
#define EIGEN_MATHFUNC_IMPL(func, scalar) \
  Eigen::internal::func##_impl<typename \
  Eigen::internal::global_math_functions_filtering_base<scalar>::type>
```

— Boost.Units applies the pattern to the inner value of a quantity type. [boost-units-sqrt]

But the fundamental approach is identical in all three.

1.1.3 Standard Library numeric types

The problem also impacts the standard library when non-arithmetic numeric types are defined, such as for simd.

Specifically, Mateusz Pusz, the author of the mp-units library and main contributor to the C++26 unit library [P3045R8], called out these very limitations at C++ on Sea 2025, specifically calling for a proposal to introduce CPOs for mathematical functions. [pusz-cpponsea2025]

The linear algebra facilities currently uses helper functions in order to allow custom arithmetic types, and it has been acknowledged in committee that having a well defined contract to define these customization points would be very helpful.

1.2 Context

The mathematical functions of C++ have been inherited from C, before the introduction of namespaces in the language and therefore lived in the global name space. With C++98, the `<cmath>` header was added, as the new correct way to access the math functions inherited from C. The `<math.h>` header was retained, though, for backward compatibility, leading to the present-day situation where many mathematical functions are

accessible in both the global namespace and `std`. C++98, C++11, and C++17 also introduced additional functions to `<cmath>`. C++29 could add to it further [P3935R1].

As a result of this evolution, these functions are unlike other parts of the standard library:

— Their names do not follow the `snake_case` pattern.
— There are several variants of each function differentiated by prefix or suffix instead of using function overloading.
— Their error handling uses `errno` instead of exceptions or the more modern `std::expected`.

Encountering this API is surprising and confusing for new C++ learners without a C background, as the justification for these divergences are largely historical.

Another consequence of this long progression is also that there is a huge body of code relying on all of these successive evolutions. These APIs are essentially frozen, as the contracts established over the years can't reasonably be invalidated.

2 Impact on the Standard

Consequently, based on the points discussed above, this proposal aims to address some of the limitations to the extensibility of the math functions in C++ while preserving existing behaviours.

Since the core problem is the absence of a standard extension mechanism, the solution must be introduced by the standard library itself. It cannot be provided by user code or third-party libraries alone.

2.1 `std::math`

Specifically, this proposal introduces a new namespace `std::math` to clearly separate the new customizable contract from the current functions inherited from C.

— Math functions in this namespace allow customization through ADL.
— Their naming strives to be consistent with modern standard library convention.
— They leverage overloading and generic programming instead of prefix and suffix.

In addition, tools offering autocompletion would also benefit from a focused, dedicated namespace narrowing the relevant options early by category rather than name only.

The introduction of a new namespace and a new contract also provides an opportunity to resolve longstanding ambiguities such as the semantics of `abs` and `norm`. The specific meaning of math functions becomes especially consequential once user-defined customization is possible.

Finally, the combination of a reduced number of function names, more predictable naming patterns, better autocompletion, and clearer semantic of the operations should help improve the teachability issues mentioned previously.

2.2 Bridging through opt-in

In addition to the new namespace, this proposal also introduces a mechanism in the `std` namespace to enable ADL resolution for types explicitly opted-in, with math functions. This is a compatibility layer with a strict contract to maintain the existing code behaviour: ADL is only allowed for explicitly opted-in types and when the alternative to ADL resolution is failing compilation.

We acknowledge the limit that existing behaviours relying on SFINAE based on mathematical functions not being resolved and applied on explicitly opted-in types might break. Opt-in should thus be exercised with caution, especially in contexts where SFINAE is conditioned on mathematical function availability.

The behaviour of this compatibility layer is deliberately different from the behaviour in `std::math`: the former prioritizes backward compatibility with explicit opt-in and ADL as a fallback; the latter prioritizes user-defined customization, without opt-in, with `std` variant as the fall-back.

2.3 Summary

This proposal doesn't require any new language feature. It would introduce a new `std::math` namespace and a compatibility layer inside of the `std` namespace, each of these two additions introducing approximately one new declaration per mathematical function, comparable in surface area to <cmath>.

3 Design Principles

3.1 Two spaces, different behaviours

The design space for this proposal is effectively split in two, between the forward-looking `std::math` layer, and the compatibility layer in `std`.

A solution in `std` only would have to compromise between extensibility and backward compatibility. Breaking the established contract there would be dangerous, and an opt-in-only solution leads to no good outcome: if adoption is low, generic code cannot rely on it for extensibility; if most types opt in, the mechanism ceases to be optional in any meaningful sense and only gets in the way.

Conversely, a new namespace alone would not help with code that calls `std` mathematical functions directly.

By combining both, the new code can benefit from the new approach in the new namespace with a new contract, while the compatibility layer in std remains available for targeted, case-by-case opt-ins.

The new `std::math` namespace splits from the mathematical functions inherited from C. It follows standard library naming patterns and leverages overloading and templates. It is intended as a native C++ library.

Compromising between C legacy and modern C++ patterns would hinder both objectives. The clear separation allows both sides to remain, each in its defined space.

3.2 New contract

The math functions in the `std::math`namespace have a new contract, independent from the `std` namespace contract.

User-customization implies a looser contract for these functions when they are dispatched to a user-defined implementation, where few aspects of such a contract can be enforced. Similarly to overloaded operators or other CPOs, the contract is owned by the implementation the call gets dispatched to.

The implementations for native types can define their own contracts, potentially forwarding the same contract as their `std` equivalent, but no such guarantee can be provided for user-defined implementations.

Since the new namespace adopts the standard library naming patterns, departing from the `<cmath>` names, several operations will have a different name between the two namespaces. In such cases, they will read as distinct functions, and the users will expect a different contract.

More importantly, with user-customization enabled, the function contract needs to be as clear as possible to avoid confusion. To that end, function names are chosen to reflect their purpose as precisely as possible.

Such an example of ambiguity arose with the `abs` and `norm` functions for the `std::complex` type, where `std::abs` was defined to be the L2 norm, and `std::norm` to be the square of the L2 norm (the squared modulus, which is not a mathematical norm). This would make it unclear how the `abs` and `norm` functions should be customized for user-defined multi-dimensional types.

3.3 CPO-like customization

Rather than a plain function template, `std::math` operations are proposed as CPO-like constructs. This design, established by the Ranges library (`std::ranges::begin`, `std::ranges::swap` etc.), offers these two advantages:

— The customization point cannot be found by ADL on user types, since it is an object rather than a function.
— The `operator()` can be constrained, making the customization point SFINAE-friendly and allowing it to be used correctly inside `requires` expressions and concepts.

3.3.1 Dispatch

The dispatch priority of these CPO-like constructs is explicitly:

1. To a member function (not subject to ADL).
2. To a free function found via ADL in the type's own namespace
3. To an equivalent `std` operation as the final fallback for all legacy types

The parameters are forwarded by forward reference.

User-defined implementation are not required to be `constexpr`, but the dispatch mechanism does not inhibit `constexpr` when it is supported.

3.3.2 Alternative to plain CPO

The term "CPO-like" is used instead of "CPO" directly to reflect the possibility of using upcoming improvement over the current CPO pattern. Specifically, if [P2806R3] (do expressions) and [P2826R2] (replacement functions) are accepted for C++29, the implementation of `std::math::sqrt` reduces significantly.

The do expression makes the ADL idiom available in expression contexts, and replacement functions provide expression-equivalence guarantees. This would allow the entire CPO to be expressed as a single declaration, without explicit dispatch concepts or a poison pill. The direction proposed here would compose naturally with these future language improvements.

3.4 Single header

All functions are defined in a single header. The specific name of this header remains to be decided.

— `<math>` is easy to remember and consistent with other header names, but might conflict with `<math.h>`.
— `<cppmath>` or `<stdmath>` would avoid that confusion.

3.5 Canonical function names

Functions in the `std::math` namespace should define a single name per operation. Introducing name variants (prefix, suffix) should be avoided. Overloading and templating are preferred instead.

This helps keeping the surface area of the library as compact as possible and allows users to reach for the mathematical operation they need in a unified way.

The purpose of each mathematical operation should be made as clear as possible from its name and signature (as exercised by the native types) to make their user-customization as unambiguous as possible.

For the `abs` example, this would mean a single `abs` operation (no `labs`, `llabs`, `imaxabs`, `fabs`, `fabsf`, or `fabsl`) which is meant to return a value of the same type as its received parameter, where all negative components of the input value have had their sign switched (component-wise abs).

`norm` would be optionally templated on an enum defining which norm is requested (L1, L2, Infinity/Chebyshev), and defaulted to L2 when omitted. An additional `norm_sq` function could be considered, returning specifically the square of the L2 norm to offer a more performant alternative where the squared value is sufficient, without sacrificing the semantic of the operation.

4 General Specifications for built-in arithmetic types

Implementations for the fundamental arithmetic types (`char`, `short`, `int`, `long`, `long long`, `float`, `double`, `long double`, and `bool` where relevant) are provided where conservatively appropriate.

Specializations for `std::complex<T>` are provided where appropriate, with semantics defined independently from their std counterparts. All other standard library and user-defined types participate through the ADL dispatch mechanism.

These Standard Library implementations abide by the following rules.

4.1 Conservative addition

`std::math` functions are not specialized for every fundamental arithmetic type through this proposal. Specifically, when the contract of a specialization is unclear and its presence not indispensable, it is deferred to a later revision.

For instance, `std::math::sqrt` return type when called on an integral type is unclear. It could be a `float`, a `double`, an `int`, or alternatively, this specialization of the operation could be templated to require an explicit return type as a template parameter. This proposal therefore chooses to leave `std::math::sqrt` undefined for integral types, keeping this design space open for future improvements.

4.2 `constexpr`

All Standard-Library-defined specialization of functions in `std::math` are `constexpr`.

4.3 `[[nodiscard]]`

All functions in `std::math` with a return type are `[[nodiscard]]` unless specified otherwise.

4.4 Error Handling

Domain errors are considered contract violations and do not constitute runtime errors. As such they don't prevent the function from being `noexcept`.

`std::math` functions define their contract explicitly. They neither `throw` nor update `errno`.

Floating-point domain errors follow `IEEE754` semantics (NaN/inf). Integral-type domain errors are Undefined Behaviour.

Standard Library specialisations for fundamental arithmetic types and `std::complex` are `noexcept`.

5 Technical Specification

6 References

[boost-units-sqrt] Boost Developers. Boost.Units: sqrt Implementation.
https://github.com/boostorg/units/blob/develop/include/boost/units/cmath.hpp

[eigen-math] Eigen Developers. Eigen: MathFunctions.h Implementation.
https://gitlab.com/libeigen/eigen/-/blob/master/Eigen/src/Core/MathFunctions.h

[mp-units] Mateusz Pusz. mp-units: Math Dispatch Implementation.
https://github.com/mpusz/mp-units/blob/b0e72810b983841b260d570b241c52586aa78999/src/core/include/mp-units/framework/representation_concepts.h#L227-L262

[nholthaus-units] Nick Holthaus. nholthaus/units Library.
https://github.com/nholthaus/units

[P2806R3] Barry Revzin, Bruno Cardoso Lopez, Zach Laine, Michael Park. 2025-01-12. do expressions.
https://wg21.link/p2806r3

[P2826R2] Gašper Ažman. 2024-03-18. Replacement functions.
https://wg21.link/p2826r2

[P3045R8] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. 2026-05-12. Quantities and units library.
https://wg21.link/p3045r8

[P3935R1] Jan Schultke. 2026-05-11. Rebasing <cmath> on C23.
https://wg21.link/p3935r1

[pusz-cpponsea2025] Mateusz Pusz. The 10 Essential Features for the Future of C++ Libraries. C++ on Sea 2025.
https://www.youtube.com/watch?v=TJg37Sh9j78&t=3158s